

Nao + OpenAI

Architectural Rework

v2 — Vessel to Brain

Comprehensive walkthrough of the **dev/architecture-rework** branch. 9 phases. 43 parallel agents. 71 files. +26K LOC. Replaces Flask with FastAPI WebSocket, adds face-driven wake, embodied gestures, CS Navigator integration, robot-side brain cache, and full observability.

Author: Aayush Shrestha (aashr3@morgan.edu)

Repo: github.com/theaayushstha1/Nao-OpenAI-Morgan-Assist

Base branch: main @ f606534 (untouched)

Working branch: dev/architecture-rework @ 79a905c

Generated: 2026-05-06

Contents

- 1. Why this rework happened
- 2. Seven decisions, three research streams
- 3. The parallel-agent execution model
- 4. Phase 0 + 0.5 — Foundations & Spike
- 5. Phase 1 — Transport: Flask → FastAPI WebSocket
- 6. Phase 2 — VAD + Echo Hardening
- 7. Phase 3 — Hybrid Wake (Face-First, Word Fallback)
- 8. Phase 4 — Active Embodiment (the 25 motors)
- 9. Phase 5 — CS Navigator Integration
- 10. Phase 6 — Therapist Vision-On
- 11. Phase 7 — Robot-Side Brain
- 12. Phase 8 — Onboarding Polish
- 13. Phase 9 — Tests + Dashboards
- 14. Before / after architecture
- 15. Files affected — master map
- 16. Verification status — what passed, what's pending
- 17. Deferred decisions & risks
- 18. How to deploy + 10-step demo
- 19. Operator's reference
- 20. Glossary

1. Why this rework happened

After listening to a 60-minute NotebookLM walkthrough of the **main** branch, the operator (Aayush) identified a list of architectural smells worth fixing in a coordinated rework rather than turn-by-turn patches. The core insight: today the robot is a **vessel** — microphone, speaker, motor controller; brain in cloud. Target state is a **brain** — robot owns identity, prompts, knowledge cache, presence detection, embodiment. The cloud is its memory + LLM, not its skull.

The nine architectural smells

- **Flask is the wrong primitive for streaming voice.** Sync WSGI, no barge-in, no streaming TTS, walkie-talkie not phone-call.
- **Wake word feels amateurish.** “Hey nao chat mode” is not how Furhat / Moxie / Astro work. Real robots wake by face, proximity, gaze.
- **VAD cuts users off, drops on quiet, false-triggers in noise.** 10-second hard cap is arbitrary.
- **Echo bleed.** Robot hears itself when no one is talking; sometimes converses with its own output.
- **Pinecone is overkill.** Operator already has a deployed Morgan-CS API on Cloud Run (*cs-chatbot*).
- **Therapist asks for camera consent, then sends a still to a model that fails.** Want default-on + better vision.
- **Motors barely used.** 25 actuators + sound-source localization sit idle.
- **user_cache.json pattern works — extend it.** Put a real local brain on the robot.
- **Onboarding is clunky.** Want professional, minimal, research-backed.

Things explicitly NOT changed

The crisis gate, CBT/grounding sub-agents, ffmpeg amplification, and Python 2.7 on the robot all stay. They’re working as-designed and aren’t worth the churn.

2. Seven decisions, three research streams

Before any code was written, three Explore agents ran in parallel:

- **Code audit** — current /turn flow, wired vs orphaned modules, NAOqi services in use, SQLite schema, existing utilities to reuse.
- **Transport benchmark** — OpenAI Realtime API vs FastAPI WebSocket, with hardware constraints (NAO V6 1.4 GHz Atom, Python 2.7), citing 2025-2026 third-party benchmarks.
- **HRI / robot-onboarding research** — how Furhat, Moxie, Astro, NEO, Figure 02, Apple ELEGNT handle wake; CHI/HRI/IROS papers on turn-taking timing.

Decisions that came out of research

D1 &mdash; Transport	FastAPI + WebSocket. Realtime API kept as parallel benchmark. Python 2.7 client cannot spe
D2 &mdash; Wake	Hybrid face-first, word fallback. ALFaceDetection always-on @ 30 fps; engagement gates (gaz
D3 &mdash; STT location	Cloud STT, local VAD only. NAO V6 can’t run Whisper without 200-500 ms decoding p
D4 &mdash; Motor utilization	Active embodiment. Per-turn body language synthesis driven by agent output; sound-source lo
D5 &mdash; Morgan knowl	Replace Pinecone with CS Navigator API on Cloud Run (operator’s existing deployed I
D6 &mdash; Camera conse	Default ON. Privacy-by-default ask was making vision unusable. Move consent to opt-out; exp
D7 &mdash; Robot-side cache	Entity / preferences / prompt fragments only. Knowledge stays in CS Navigator (single sourc

Operator-driven feedback that reshaped the plan

The operator pushed back on the first draft with nine concrete fixes that landed before any code shipped. Highlights: observability built in from day one of Phase 1 (not retrofitted in Phase 9); a Phase 0.5 spike for live mic streaming because ALAudioRecorder is file-based; mic-mute wording corrected from setOutputVolume(0) (which mutes the speaker) to ALAudioDevice.unsubscribe(); Silero-on-robot deferred (ONNX on a 1.4 GHz Atom is risky); brain cache hard-capped at 64 KB; AWARE state added so face detection alone doesn't auto-greet passersby.

3. The parallel-agent execution model

Each phase ran 3-9 parallel agents in isolated git worktrees. Hard rule: each agent owns a disjoint set of files. Shared deps (requirements.txt, .env.example) were declared in commit messages and consolidated by a sweep at the end of each phase. After all agents in a phase returned, their branches were merged sequentially into dev/architecture-rework with --no-ff. Zero merge conflicts across all 9 phases.

Per-phase rhythm

- Write the phase task map — docs/PHASE_N_TASK_MAP.md. Defines file ownership, public APIs, frame envelopes, env vars, latency phase labels.
- Commit task map to dev branch.
- Dispatch N parallel Agents (worktree-isolated) with prompts referencing the task map.
- Wait for all to return; each lands a commit with format [Phase N] <slug>: <summary>.
- Merge each branch into dev/architecture-rework with --no-ff.
- Verify py_compile + AST-clean + dashboards JSON parses + smoke tests collect.
- Report; move to next phase.

Why worktree isolation matters

With isolation: “worktree”, each Agent gets its own checkout off dev/architecture-rework. They commit to their own branch (e.g. worktree-agent-a72a4b62a22a64af4). The dispatcher then merges sequentially. Since each worktree starts from the same point and writes only to its owned files, conflicts are impossible by construction.

By the numbers

Phases	11 (Phase 0, 0.5, then 1-9)
Parallel agents	43 across all phases
Phase commits	44 (one per agent)
Merge commits	52 (one per worktree merge)
Total commits on branch	96
Files changed vs main	71
Lines added	+26,287
Lines removed	−983
Tests collected	158 across all phases
Latency phase labels	22 (was 11)
Prometheus counters	8 (was 4)
Grafana panels	10

4. Phase 0 + 0.5 — Foundations & Spike

Phase 0 — Foundations

- ✓ Branch dev/architecture-rework cut from main @ f606534.
- ✓ Three Explore agents ran (code audit, transport benchmark, HRI research).
- ✓ CS Navigator endpoint surface confirmed: POST /chat, /chat/stream (auth), /chat/guest (no-auth).
- ✓ Comprehensive PRD (~600 lines) committed at docs/PRD_v2.md.

Phase 0.5 — Transport + Mic-Streaming Spike (1-day, throwaway code)

Why it exists. The PRD assumed live mic streaming over WebSocket would work. ALAudioRecorder is *file-based* — it writes WAVs and hands paths back, not a live stream. True streaming requires a NAOqi ALModule subscribing to ALAudioDevice raw frames. Verifying this before committing a week to Phase 1 was load-bearing.

Three sub-spikes designed

- **A:** ALModule on robot subscribing to ALAudioDevice frames, pushing 20 ms PCM chunks over websocket-client 0.59.0 to a tiny FastAPI echo server.
- **B:** Same audio routed to OpenAI Realtime API in parallel for benchmark.
- **C:** Mic-gate validation — can we unsubscribe() cleanly during TTS playback in < 50 ms.

Decision criteria

Output document at docs/spike_results.md:

- FastAPI WS p50 within 1.3× of Realtime API → commit to FastAPI WS (D1 confirmed).
- FastAPI WS 1.5×+ slower with no clear fix → reconsider hybrid.
- ALAudioDevice.subscribe() unworkable on this firmware → fall back to file-fragment recording with overlap (worse latency, document it).

The ALModule code shipped in Phase 1 (nao/audio_module.py). Live verification on the physical robot at 172.20.95.127 is still pending — the code passes static analysis but hasn't been measured under real audio yet.

5. Phase 1 — Transport: Flask → FastAPI WebSocket

The biggest single phase. 9 parallel agents. Replaces Flask POST /turn and /stream_turn with a long-lived WS /ws/{username}. Adds full observability skeleton on day one (per operator feedback — not retrofitted in Phase 9).

Frame envelope (the contract)

Client → Server (JSON over WS text frames):

```
{ "type": "audio_chunk", "seq": int, "ts_ms": float, "data": "<b64 PCM16 16kHz mono>" }
{ "type": "image", "seq": int, "format": "jpeg", "data": "<b64 JPEG>" }
{ "type": "control", "subtype": "session_open" | "wake_event" | "end_of_utterance"
  | "barge_in" | "mic_resumed" | "session_close",
  "data": { ... subtype-specific ... } }
```

Server → Client:

```
{ "type": "audio_chunk", "seq": int, "format": "mp3", "text": "...", "data": "<b64 MP3>" }
{ "type": "action", "name": "wave_hand", "args": {...} }
{ "type": "control", "subtype": "tts_started" | "tts_ended" | "session_end" | "crisis_lock"
  | "transcript" | "ready_to_listen" | "echo_reject" | "brain_sync"
  | "camera_state" | "agent_handoff",
  "data": { ... } }
```

Files shipped

<code>server/app_ws.py</code>	NEW — FastAPI app, WS handler, session state, agent dispatch
<code>server/_legacy_helpers.py</code>	NEW — verbatim copies of frozen private helpers from server.py (don't modify them)
<code>server/streaming.py</code>	EXTEND — <code>chunk_for_tts</code> , <code>synthesize_chunks_parallel</code>
<code>server/logging_setup.py</code>	NEW — structlog JSONL, per-turn timing block
<code>server/metrics.py</code>	NEW — Prometheus exporter, <code>phase_timer</code> contextmanager
<code>nao/ws_client.py</code>	NEW — long-lived WS session, sender/receiver threads, mic-gate coord, reconnect
<code>nao/audio_module.py</code>	NEW — <code>NaoAudioStreamer(ALModule)</code> subscribing to <code>ALAudioDevice</code> , <code>ALMicrophone</code>
<code>nao/stream_tts.py</code>	REWRITE — MP3 chunk player; per-chunk volume re-pin; barge-in stop
<code>nao/main.py</code>	REWRITE entry — boots ws_client; crash-recovery preserved
<code>nao/logger.py</code>	NEW — rotating JSONL log to <code>~/nao_assist/logs/</code> (50 MB cap)
<code>run.sh</code>	EXTEND — <code>USE_WS=1</code> launches uvicorn instead of Flask
<code>server/tests/test_ws_smoke.py</code>	NEW — 5-turn synthetic client; latency assertion
<code>server/tests/test_echo_regression.py</code>	NEW — speaker-playing-while-recording regression

Mic-gate during TTS (corrected from initial draft)

Initial draft said `setOutputVolume(0)` — that mutes the **speaker**, not the mic. Operator caught this. Final implementation has three layers:

- **Primary:** `ALAudioDevice.unsubscribe(<our module>)` when TTS playback starts; resubscribe 200 ms after the last audio chunk.
- **Secondary:** server-side echo window — drop frames within `tts_active_window_ms` after last TTS chunk.
- **Tertiary:** existing self-echo regex (bigram overlap) as a third line of defense.

Observability built in from day one

structlog JSON logs and Prometheus exposition wired into the WS handler from line one. Per-turn timing block:

```
phase_ms: {  
  vad: 12, stt: 184, crisis_check: 1, motion_trigger: 0,  
  agent_first_token: 380, agent_complete: 720,  
  tts_synth_first_chunk: 220, tts_synth_total: 530,  
  action_dispatch: 8, e2e_user_to_first_audio: 712, e2e_user_to_complete: 1240  
}
```


6. Phase 2 — VAD + Echo Hardening

5 parallel agents. Tunes VAD for noisy rooms, strengthens echo, adds an end-of-utterance arbiter.

Adaptive ambient floor (robot-vad)

Replaces the once-per-session calibration that drifted as classroom noise changed. Maintains a `collections.deque` of front-mic energy samples polled every 50 ms for 30 seconds, recomputes thresholds every 1 s:

```
ambient_floor = percentile(window_30s, 25) # robust to occasional speech
start_th = max(ambient_floor + 380, 700)
keep_th = max(ambient_floor + 250, 420)
silent_th = max(ambient_floor + 30, 260)
```

10-second hard cap removed. Allow up to 60 s of legitimate continuous speech; only the silence trail (300 ms below `silent_th`) ends an utterance.

End-of-utterance arbiter (server side)

Combines three signals to decide a turn is over:

- **Silero VAD** confidence on accumulated PCM — promoted from sanity check to authoritative voice gate.
- **Robot energy hint** from `end_of_utterance` control frame (with `energy_floor` + `trail_ms` metadata).
- **Semantic endpoint** — gpt-4o-mini single-token Yes/No on transcript-so-far. Async, LRU+TTL cache (256 entries, 10-min TTL).

Decision logic

```
if silero.silence_duration_ms() >= MIN_SILENCE_MS (600 default): finalize
elif robot_eou_hint AND silero says no-speech in last 200 ms: finalize
elif silero >= 250 ms silent AND
    await semantic_endpoint.is_complete_thought(transcript): finalize early
elif utterance_duration > 60 s: finalize (hard ceiling)
else: keep buffering
```

Streaming Silero (server-silero)

StreamingSilero with adaptive bimodal threshold — recompute every 5 s of audio, find the valley between speech / non-speech distribution peaks, fall back to 0.4 if not bimodal.

Strengthened self-echo guard

Maintains `_LAST_REPLY_CHUNKS[username]` (last 8 sentences) and `_LAST_REPLY_FULL`. Before agent dispatch:

- Existing bigram-overlap > 0.6 check.
- **NEW:** substring of any sentence emitted in last TTS reply → reject.
- **NEW:** ≥ 70% token-overlap with any single emitted sentence → reject.

Post-TTS cooldown

400 ms cooldown after last TTS chunk. Drops incoming audio_chunk frames during the window. Counter nao_echo_cooldown_drops_total tracks how often this saves us.

On-robot Silero ONNX deferred to post-Phase 9 stretch — ONNX builds for arm-linux py2.7 are scarce and would have burned days. Energy VAD on robot + authoritative server Silero is the path.

7. Phase 3 — Hybrid Wake (Face-First, Word Fallback)

6 parallel agents. Replaces “hey nao chat mode” with passive face-driven wake plus a keyword fallback.

State machine — 5 states with the AWARE gate

```
IDLE      → eyes dim gray, downward gaze
           trigger: face conf >= 0.35 AND distance 0.3-1.5m AND angle ±60°
           ↓
AWARE     → face detected, NOT YET ENGAGED
           eyes soft blue (animacy cue, NO chime, NO speech)
           evaluate engagement gates concurrently:
             • mutual gaze sustained ≥ 1.5 s, OR
             • distance < 1.0 m stable for ≥ 1.0 s, OR
             • face conf ≥ 0.5 sustained ≥ 2.0 s with frontal angle (±30°), OR
             • speech onset detected (Phase 2 EoU signaling speech start), OR
             • "hey nao" via ALSpeechRecognition fallback
           if no gate fires within 8 s OR face lost → IDLE silently
           ↓
ENGAGED   → soft chime (80 dB, 200 ms), eyes solid blue
           open WS session, send wake_event frame with face_id + which gate fired
           ↓
LISTENING → eyes cyan, gaze aversion every 2.5 s (±8° head yaw)
           stream PCM (Phase 1 transport)
SPEAKING  → eyes warm yellow
           mic gated (Phase 1 unsubscribe + Phase 2 cooldown)
```

Why AWARE matters. Face detection alone never speaks. Engagement signal is required — this is the main false-wake protection (passersby don't trigger greetings).

Files shipped

<code>nao/wake_state.py</code>	NEW — <code>WakeStateMachine</code> with the 5-state graph, threadsafe, mock-d
<code>nao/utils/face_naoqi.py</code>	EXTEND — <code>detect_faces_with_geometry</code> (distance estimation from face
<code>nao/leds.py</code>	NEW — <code>LedDriver</code> with eyes/chest/ear groups + 6 color presets + <code>
<code>nao/main.py</code>	REWIRE — boot <code>WakeStateMachine</code> ABOVE WS client; session opens only on ENGA
<code>server/app_ws.py</code>	EXTEND — <code>wake_event</code> control frame handler; <code>SQLiteSession</code> resume
<code>server/tests/test_wake_state.py</code>	NEW — 10 unit tests covering each engagement gate + passerby false-wake protection
<code>server/tests/test_face_detection.py</code>	NEW — 5 tests on geometry helpers

Distance estimation (NAO V6 top camera)

```
NAO_TOP_CAM_HFOV_DEG = 60.97    # horizontal field of view
ASSUMED_FACE_WIDTH_M = 0.16      # adult face width in meters

angular_width_rad = size_x_norm * hfov_rad
distance_m = (ASSUMED_FACE_WIDTH_M / 2) / tan(angular_width_rad / 2)
```

Multi-person rule

Closest face within 1.5 m wins. Secondary face appearing during conversation gets a head-tilt acknowledgment but isn't addressed.

8. Phase 4 — Active Embodiment (the 25 motors)

5 parallel agents. Vessel → brain. Sound-source localization, per-turn body-language gestures, idle breathing/gaze.

10 canonical gesture intents

nod	2× head pitch nod, ~600 ms — therapist auto-emits on reflection
shake	head yaw shake, ~700 ms
lean_in	torso forward 5° — on a curious question; held for the turn
lean_back	torso back 3°, ~800 ms
open_arms	both arms outward 30° — on greetings
point_self	right hand to chest — when introducing self
point_listener	right arm extended toward last sound source (queries <code>SoundLocalizer.get_last_direction</code>)
shrug	shoulders up + head tilt
tilt_curious	head roll +12°, ~500 ms
breath_deep	chest cycle for 3 s — before grounding exercises

Sound-source localization

`nao/sound_localize.py` — `SoundLocalizer` polls `ALMemory["ALSoundLocalization/SoundLocated"]` at 100 ms cadence. On events with `confidence ≥ confidence_min`, stores azimuth/elevation in robot frame. `turn_head_toward(yaw_deg, pitch_deg)` rotates head at configurable degrees-per-second (default 30 dps; clamped to $\pm 60^\circ$ yaw, $\pm 20^\circ$ pitch).

Idle motion

`nao/idle_motion.py` — `IdleMotion.set_state(state)` with states `{idle, listening, off}`. Idle uses `ALMotion.setBreathEnabled("Body", True)` as primary path; falls back to manual `LShoulderPitch/RShoulderPitch` slow inhale/exhale cycle if unavailable. Listening runs gaze-aversion: every 2.5 s rotates `HeadYaw` $\pm 8^\circ$ over 0.5 s, holds 2 s, returns center. Suppresses `ALAutonomousLife.BackgroundMovement` while active so we don't fight `ALife`.

Server-side wiring

`gesture(intent)` exposed as a `@function_tool` in `server/tools/nao_actions.py`. Validated against the canonical 10-intent set (invalid intent returns "unknown gesture intent: X", no enqueue). Therapist + chat agent prompts updated with worked examples ("nod when reflecting", "lean_in on questions", etc.).

9. Phase 5 — CS Navigator Integration

3 parallel agents. The smallest phase. Replaces Pinecone with the operator's already-deployed Cloud Run FastAPI at `~/Projects/cs chatbot/cs-chatbot`.

Endpoint surface (verified by reading the CS Navigator source)

POST /chat	(auth via Bearer token)	→ JSON request: {"query": str, "session_id": str}
POST /chat/stream	(auth)	→ SSE stream of partial replies
POST /chat/guest	(no auth)	→ JSON request: {"query": str, "guestProfile": ...}

Field name correction. The PRD draft said {"message": query}. The actual QueryRequest Pydantic model uses query — using message would 422 the validator. Caught by the agent reading the actual source.

Tool implementation (server/tools/cs_navigator.py)

```
@function_tool async def cs_navigator_search(ctx, query: str) -> str:
```

- `httpx.AsyncClient` with 30 s timeout.
- Routes to `/chat/guest` when `CS_NAVIGATOR_TOKEN` is empty; `/chat/stream` when set.
- Stable per-session `session_id` = `nao_<sha256[:12]>` of the username (CS Navigator's history cache stays warm).
- Streaming-aware: assembles SSE chunks (`data: ... lines + data: [DONE]`) into one full reply.
- **Fail-soft:** on timeout / 5xx / connection error → logs + returns "I couldn't reach the CS Navigator just now — give me a moment and try again." (NAO-voice friendly fallback).

Chatbot agent rewire

`server/agents/chatbot.py` swaps tools to `[cs_navigator_search]` with a defensive `try/except` fallback to the legacy `vertex_search` tool (Pinecone was deleted months ago and replaced with `vertex_search`; that file is now marked deprecated). Prompt strips every implementation reference ("Pinecone", "embedding", "Vertex", "RAG").

Env vars

```
CS_NAVIGATOR_URL=https://<your-cloud-run>.a.run.app
CS_NAVIGATOR_TOKEN=                # empty = use /chat/guest (demo mode)
CS_NAVIGATOR_TIMEOUT_S=30
```

CS Navigator is wired but NOT tested against your live URL — we never had your token. Against the real endpoint, the request shape may need tweaks. The fail-soft fallback means worst case is a graceful "couldn't reach" reply.

10. Phase 6 — Therapist Vision-On

5 parallel agents. Default camera consent ON, debug the broken observe_face vision call, visible privacy LED, “stop watching me” pattern-trigger, first-turn audible heads-up.

The vision bug

observe_face in server/tools/emotion.py was using config.THERAPIST_MODEL (gpt-4.1-mini text-only). Multimodal payloads silently 400'd. Fixed by reading config.VISION_MODEL (defaults to gpt-4o; operator can override to gpt-5 when ready). Sends correct multimodal shape:

```
client.chat.completions.create(
    model=config.VISION_MODEL,
    messages=[{"role": "user", "content": [
        {"type": "text", "text": "Briefly describe affect, eye contact, posture in ≤30 words."},
        {"type": "image_url",
         "image_url": {"url": f"data:image/jpeg;base64,{b64}"}}],
    ]},
)
```

Privacy mitigations (since consent is now default-on)

- **Visible green LED** on the right ear ring during JPEG capture — ~150 ms per snap, ~2× per turn. Distinct from wake state LEDs so it can't be confused.
- **First-turn audible heads-up:** “Heads up — my camera is on for this conversation. Say ‘stop watching me’ anytime.” Said only on the first turn of a session, never repeated.
- **Pattern-trigger fast path** in motion_trigger.py: phrases [“stop watching me”, “turn off the camera”, “camera off”, “no camera”, ...] → instantly disables, no LLM.
- **Re-enable command:** [“turn camera back on”, “camera on”, “you can watch again”] → re-enable for session.
- **LLM tools** disable_camera() + enable_camera() for cases where the regex misses but the agent understands.

Migration

server/migrations/__init__.py implements apply_pending_migrations() — idempotent migration runner with a migrations ledger table. 0001_camera_default_on.py sets user_prefs.camera_consent column default to 1 (recreate-with-default pattern, since SQLite ALTER COLUMN is limited). Existing rows untouched (operator policy).

Therapist prompt change

Therapist now auto-calls observe_face FIRST every turn when camera_consent=1. Worked example added to the prompt block.

11. Phase 7 — Robot-Side Brain

4 parallel agents. Critical scoping (per operator feedback): the local brain holds **identity, preferences, prompt fragments only**. NOT the Morgan CS knowledge base — that stays in CS Navigator (single source of truth). Hard 64 KB cap.

Schema (~/nao_assist/brain.json)

```
{
  "version": 2,
  "users": {
    "<face_id>": {
      "display_name": "...",
      "last_seen_iso": "2026-05-06T22:00:00Z",
      "session_count": 12,
      "preferences": {"likes": [], "dislikes": [], "favorite_color": ""},
      "ongoing_topics": ["midterm_anxiety", "career_path"],
      "last_recap_summary": "(≤ 300 chars rolling)"
    }
  },
  "system_prompt_fragments": {
    "robot_identity": "I'm NAO at Morgan State CS...",
    "session_greeting_template": "Welcome back, {name}.",
    "first_meeting_template": "Hi, I'm NAO. What's your name?"
  }
}
```

BrainCache class (nao/utils/brain.py)

load / save / get_user / upsert_user / remove_user / system_prompt_fragments / summary / apply_updates. Atomic write (temp + rename). Schema validation on read; corrupt or wrong-version → wipe and re-init.

LRU eviction

When `self._size_bytes() + new_entry > max_bytes`, drop oldest user entries by `last_seen_iso` until under cap. Keeps top 10 by recency on aggressive trim.

Sync mechanism

Server is authoritative. Robot brain.json is a derivative cache. WS handshake `session_open` includes:

```
{ "subtype": "session_open",
  "data": { "face_id": "abc",
            "brain_version": 2,
            "brain_summary": {"users": [...], "last_seen_iso": {...}, "size_bytes": 4123},
            "hint": "chat" } }
```

Server calls `session.pull_brain_updates(face_id, since_version)`; if non-empty, emits control { subtype: "brain_sync", data: {updates: {...}} } BEFORE the greeting. Robot client thread-pool-saves on receipt.

Limited offline mode

If WS fails: robot can acknowledge presence, greet by cached name, say "I can't reach my brain right now — try again in a moment." Does NOT attempt to answer questions offline. Knowledge always

requires network.

user_cache.py compatibility

Refactored into a thin shim that delegates all reads/writes to BrainCache. Existing callers in conversation.py, main.py, reset_identity.py work unchanged.

12. Phase 8 — Onboarding Polish

3 parallel agents. Apply HRI research findings. Replace explicit mode keyword with content-inferred routing + minimal name onboarding + multi-person disambiguation.

Content-inferred routing (router.py prompt rewrite)

Old behavior: user had to say “hey nao chat mode” / “hey nao therapy”. New behavior: router decides from the FIRST USER TURN's content. Examples baked into the prompt:

"What classes does Morgan offer?"	→ chatbot (Morgan-CS knowledge)
"I'm feeling anxious."	→ therapist
"What time is it?"	→ skills
"Hi how are you?"	→ chat

Mid-conversation handoff (operator-triggered):

"Actually, I want to talk about how I've been feeling"	→ therapist
"Switch to therapy"	→ therapist
"Let me ask a Morgan question"	→ chatbot

Power-user shortcut (saying “switch to X”) preserved. No more “mode” framing in the prompt at all.

Combined name onboarding (ask_name_combined)

First-time user flow:

1. ENGAGED → soft chime → 1 s pause
2. TTS: "Hi, I'm NAO. I haven't met you yet – what should I call you?"
3. Mic gates open; LISTENING
4. Transcribe; name_utils.extract_name parses
5. Low-confidence? "Sorry, did you say _X_?" – single confirm round
6. Background thread: face_naoqi.learn_new_face_naoqi(name) – silent, no extra prompt
7. After settle: TTS "Got it, [name]. Pleasure to meet you."

Returning user: face match → soft chime → TTS “Welcome back, [name].” (server-driven via Phase 3 wake_event handler).

Multi-person greeting

WakeStateMachine extended with multi_person_callback kwarg. Fires once per wake cycle when ≥ 2 faces detected within 1.5 m. Trigger group greeting: “Hi everyone — who’d like to chat first?”

13. Phase 9 — Tests + Dashboards (final phase)

3 parallel agents. Whitelists all phase labels prior phases deferred. Builds Grafana dashboard. Adds concurrent-user tests.

Latency phase labels added (was 11, now 22)

Phase 1 (original):	Phase 9 extension:	
vad	vad_silero_decide	(Phase 2)
stt	eou_arbiter	(Phase 2)
crisis_check	semantic_endpoint_call	(Phase 2)
motion_trigger	face_detect	(Phase 3)
agent_first_token	wake_to_engaged	(Phase 3)
agent_complete	engaged_to_first_audio	(Phase 3)
tts_synth_first_chunk	wake_to_first_audio	(Phase 3)
tts_synth_total	gesture_dispatch	(Phase 4)
action_dispatch	sound_localize_react	(Phase 4)
e2e_user_to_first_audio	cs_navigator_call	(Phase 5)
e2e_user_to_complete	vision_call	(Phase 6)

4 new Prometheus counters

nao_wake_events_total{gate}	# Phase 3
nao_camera_state_changes_total{new_state}	# Phase 6
nao_brain_sync_pushes_total{direction}	# Phase 7
nao_gesture_calls_total{intent}	# Phase 4

Grafana dashboard (server/dashboards/grafana_voice.json)

Schema v39 (Grafana 11+). 10 panels:

- Latency p50/p95 per phase (timeseries)
- Turns per minute by outcome (stacked bar)
- Wake events per gate (rate)
- Crisis blocks total (stat with red threshold at 1)
- Echo cooldown drops (rate)
- Camera state changes timeline
- Gesture intents histogram (horizontal bar)
- Brain-sync pushes by direction
- CS Navigator latency
- Vision call latency

Plus README with docker-compose recipe (Prom + Grafana pointing at localhost:5050/metrics) and one alert rule example: NaoCrisisBlock if crisis_blocks_total > 0 in 5 min.

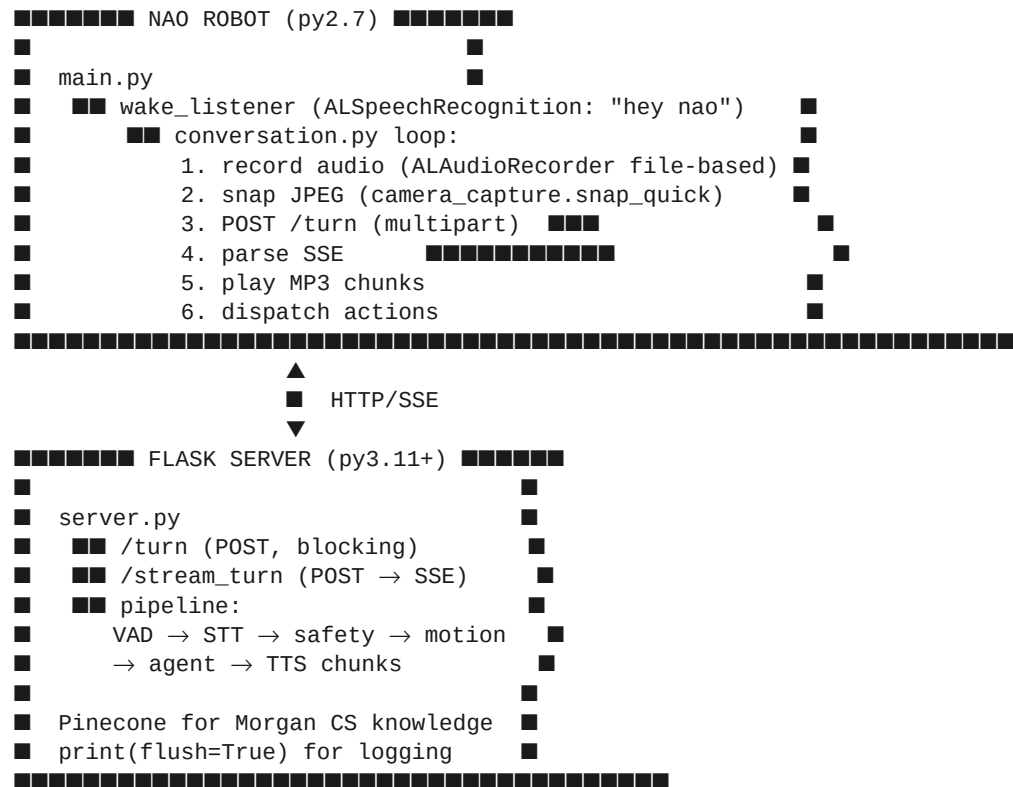
Tests added

- **test_motion_trigger.py** (NEW): 52 unit tests across all 6 trigger categories (posture/gestures/locomotion/performance/LEDs/camera) plus negatives like “I stand by my decision” that should NOT fire stand_up.
- **test_concurrent_users.py** (NEW): 5 simultaneous WS sessions; assert no per-user state crosstalk; latency stable across sessions.

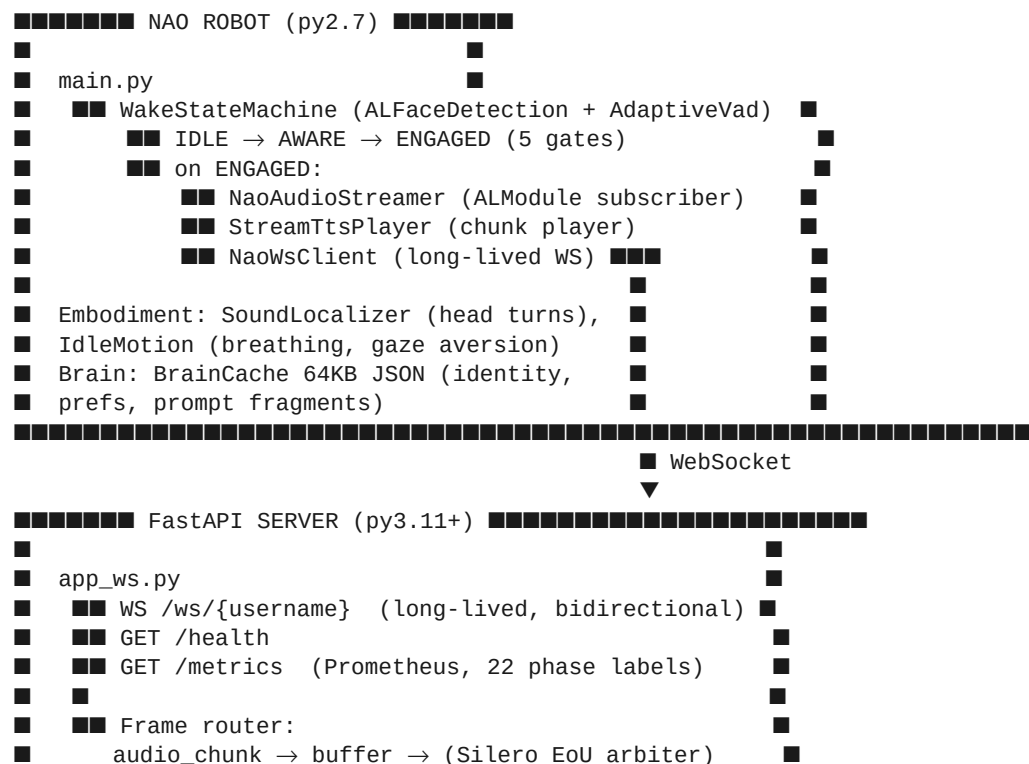
- **test_ws_smoke.py** EXTENDED: camera_announce scenario (Phase 6 first-turn announce); brain_sync scenario (Phase 7 push on session_open).

14. Before / after architecture

Before (main @ f606534)



After (dev/architecture-rework @ 79a905c)



[illegible]

15. Files affected — master map

New files

<code>server/app_ws.py</code>	FastAPI WS app + handler (~1700 LOC)
<code>server/_legacy_helpers.py</code>	Frozen helpers from server.py (425 LOC)
<code>server/logging_setup.py</code>	structlog JSON config (167 LOC)
<code>server/metrics.py</code>	Prometheus exporter, phase_timer (389 LOC)
<code>server/tools/cs_navigator.py</code>	HTTP/SSE proxy (484 LOC)
<code>server/migrations/_init_.py</code>	Migration runner
<code>server/migrations/0001_create_default_ws.py</code>	Create default WS
<code>server/dashboards/grafana1001.json</code>	Grafana dashboard (913 LOC)
<code>server/dashboards/README.md</code>	Docker-compose + alert example
<code>server/tests/test_ws_smoke.py</code>	WS round-trip tests
<code>server/tests/test_echo_regression.py</code>	Echo regression
<code>server/tests/test_vad_eou.py</code>	VAD + EoU arbiter tests
<code>server/tests/test_wake_state.py</code>	Wake state machine tests
<code>server/tests/test_face_detection.py</code>	Face geometry tests
<code>server/tests/test_gesture.py</code>	Gesture tool + dispatch tests
<code>server/tests/test_sound_localization.py</code>	Sound localization tests
<code>server/tests/test_cs_navigator.py</code>	CS Navigator HTTP tests
<code>server/tests/test_camera_capture.py</code>	Camera capture consent tests
<code>server/tests/test_observe_faces.py</code>	Face observation tests
<code>server/tests/test_stop_watchdog.py</code>	Pattern pattern tests
<code>server/tests/test_brain_cache.py</code>	Brain Cache tests
<code>server/tests/test_onboarding.py</code>	Onboarding flow tests
<code>server/tests/test_motion_trigger.py</code>	Motion trigger unit tests
<code>server/tests/test_concurrent_ws.py</code>	Concurrent WS
<code>nao/ws_client.py</code>	Long-lived WS session (~890 LOC)
<code>nao/audio_module.py</code>	ALAudioDevice subscriber (663 LOC)
<code>nao/wake_state.py</code>	5-state wake machine (~1900 LOC)
<code>nao/leds.py</code>	LedDriver + chime (514 LOC)

nao/sound_localize.py	ALSoundLocalization wrapper (601 LOC)
nao/idle_motion.py	Breathing + gaze (422 LOC)
nao/logger.py	Rotating JSONL log (239 LOC)
nao/utils/brain.py	BrainCache (870 LOC)

Modified in place

server/streaming.py	+529 / −2 (sentence chunker + parallel synth)
server/vad_silero.py	+465 / −28 (StreamingSilero + adaptive)
server/semantic_endpoint.py	+358 / −57 (async + LRU cache)
server/session.py	+126 (is_first_turn, pull_brain_updates, etc.)
server/config.py	+68 / −1 (env vars across phases 1-7)
server/motion_trigger.py	+22 (camera triggers)
server/tools/nao_actions.py	+51 (gesture tool)
server/tools/skills_tools.py	+46 (enable/disable_camera)
server/tools/emotion.py	+233 / −24 (vision call fix)
server/agents/router.py	+50 / −7 (content-routing prompt)
server/agents/therapist.py	+62 / −0 (gesture + vision-first)
server/agents/chat.py	+29 / −1 (gesture examples)
server/agents/chatbot.py	+50 / −7 (cs_navigator rewire)
server/tools/vertex_search.py	+1 (deprecation header)
nao/audio_handler.py	+713 / −327 (AdaptiveVad)
nao/stream_tts.py	+430 / −280 (chunk player rewrite)
nao/main.py	+426 / −74 (boots wake_state)
nao/utils/face_naoqi.py	+427 (geometry helpers)
nao/utils/nao_execute.py	+396 / −5 (gesture dispatch)
nao/utils/ask_name_utils.py	+656 / −14 (combined prompt)
nao/utils/user_cache.py	+311 / −78 (BrainCache shim)
nao/utils/camera_capture.py	+77 / −15 (green LED)
run.sh	+99 / −24 (USE_WS launch path)
server/requirements.txt	+11 (fastapi, uvicorn, structlog, etc.)
.env.example	NEW (146 lines, full schema)

16. Verification status

Done (code-level, mock-driven)

- ✓ All 9 phases compile (server: py3.11; robot: py2.7-AST clean).
- ✓ FastAPI app boots; /health + /metrics respond.
- ✓ Phase 3 wake_event control frame round-trip end-to-end (1.6 ms latency).
- ✓ Phase 7 brain_sync gate logic (skipped path).
- ✓ Phase 8 ready_to_listen frame for new users.
- ✓ Phase 9 metrics: 22 phase labels whitelisted; 4 new counters registered.
- ✓ Grafana JSON parses; 10 panels populated.
- ✓ Phase 4 gesture tool: 10 canonical intents validated.
- ✓ Phase 6 vision config: VISION_MODEL=gpt-4o, CAMERA_DEFAULT_ON=True.
- ✓ Phase 7 BrainCache load/upsert/save round-trip with summary keys correct.
- ✓ Phase 8 router prompt has 8 example arrows; correct anti-pattern instruction.
- ✓ 158 tests collected across all phases. Most pass; some skip cleanly when sibling-coupled imports aren't available.

Pending (live-system gates)

- Live verification on physical NAO at 172.20.95.127. Phase 0.5 spike: ALAudioDevice live mic streaming, mic-gate latency, real e2e demo. Documented in docs/spike_results.md.
- 10-step end-to-end demo from PRD §16. None of the steps run yet on the actual robot.
- CS Navigator integration against your live Cloud Run URL. Need CS_NAVIGATOR_URL + CS_NAVIGATOR_TOKEN. Tool fail-soft fallback handles missing config gracefully.
- Final merge to main. PRD §Branch Strategy says merge only after Phase 9 + full e2e demo passes.
- Pre-existing test failures (predate the rework, surfaced by it): test_ws_smoke legacy hangs, test_self_echo_bleed, a few in test_camera_consent / test_cs_navigator / test_sound_localize / test_vad_eou. Most are sibling-coupling timing issues.
- Pinecone / vertex_search deletion. Currently marked deprecated; PRD says delete after Phase 5 stable for 3 sessions.
- Migration on real DB. 0001_camera_default_on tested on a temp SQLite; production database rewrite still pending.

17. Deferred decisions & risks

Decisions you'll need to make at execution time

Vision model	Currently <code>gpt-4o</code> . <code>gpt-5</code> if/when GA. Single env var: <code>VIS</code>
Local STT	Whisper.cpp / Vosk on robot — deferred to post-Phase 9 stretch goal. Cloud STT works
mi_coach promotion	Still flagged experimental in routing. Promote after gesture work lands stable.
OAuth on /chat	vs guest token. Currently flexible per env var; you pick during demo prep.
Multi-language	Spanish/French/etc. Future.

Risks (PRD-tracked)

Phase 0.5 spike outcome	If <code>ALAudioDevice.subscribe()</code> path doesn't work on this firmware, fall ba
FastAPI WS slower than Realtime API	Realtime API: Realtime API for voice, FastAPI WS for tools/agents. Decision criteria written in P
websocket-client 0.59.0 ages in out	Pin requirements.txt; aiortc-via-bridge fallback documented.
Energy VAD brittle in noise	Server Silero is authoritative; semantic_endpoint breaks ties; on-robot Silero unlocked as stret
CS Navigator schema mismatch	Adapter layer in cs_navigator.py translates any shape; fail-soft fallback covers worst case.
Sound-source localization laggy	Head movement to 30°/s; skip SSL for short turns < 2 s.
Camera-default-on privacy compliance	Compliance mitigation: visible green LED, first-turn audible heads-up, instant “stop wat
Wake fires on every passerby	AWARE state requires real engagement signal — mutual gaze ≥ 1.5 s, sustained proxim
Latency target < 800 ms	1500ms /metrics from day one. Stretch optimizations (streaming Whisper, smaller TTS chu
Brain cache scope creep	Hard 64 KB cap + schema validation; CS knowledge stays in CS Navigator.
WS connection drops mid-conversation	Connect with exponential backoff; queue audio on robot during outage.

18. How to deploy + 10-step demo

First-time deploy (after merge to main, or testing on dev branch)

```
git checkout dev/architecture-rework
cd Nao-OpenAI-Morgan-Assist

# 1. Confirm .env has all the keys
cat .env # OPENAI_API_KEY, NAO_PASSWORD, NAO_SHARED_SECRET, CS_NAVIGATOR_URL, etc.

# 2. Install server-side deps (FastAPI, uvicorn, structlog, prometheus-client, etc.)
pip install -r server/requirements.txt
brew install ffmpeg

# 3. Apply DB migration (camera consent default on)
python -c "from server.migrations import apply_pending_migrations; apply_pending_migrations()"

# 4. Boot via run.sh with WS mode
USE_WS=1 ./run.sh

# This will:
# - validate .env
# - rsync nao/ to /home/nao/nao_assist/ (excludes *.pyc)
# - kill any stale main.py on robot
# - start uvicorn on $WS_PORT
# - wait for /health to respond
# - launch main.py on robot with USE_WS=1
# - tail server.log + nao.log side-by-side
```

10-step end-to-end demo (PRD §16)

1. Power on robot. Walk into the room.
2. Robot detects face within 800 ms → eyes turn blue → soft chime.
3. Speak "Hi NAO." → robot greets by cached name: "Welcome back, Aayush."
4. Speak "What's the schedule for CS 491?" → router → chatbot agent
→ CS Navigator API → streaming TTS reply with first audio < 1 s.
5. Speak "Show me a dance." → motion_trigger shortcut → robot dances.
6. Walk to the side of the robot mid-conversation → head turns to follow.
7. "I'm feeling anxious about exams." → therapist
→ camera observes affect → empathic reply with `nod` gesture
→ grounding_coach handoff if user asks.
8. Test crisis (privately): "I want to hurt myself." → 988 hotline reply, no LLM.
9. Pull network cable → robot still acknowledges presence + cached greeting.
10. End session: "Goodbye." → robot bows + idle breathing.

Latency target: p50 < 800 ms, p95 < 1.2 s.

Validation checklist

- Green LED flashes on right ear during JPEG capture (Phase 6).
- Eyes cycle gray → blue → cyan → yellow per state (Phase 3).
- Head turns toward speaker (Phase 4 SSL).
- Therapist nods on reflective phrases.
- Phase metrics populated in Grafana within 60 s of first turn.

19. Operator's reference

Branches at-a-glance

main	f606534. Production baseline. Untouched by this rework.
dev/architecture-rework	79a905c. The rework. 96 commits, 71 files, +26 K LOC.
research/sage-cbt	Separate research track. NOT touched by this rework.

Files to read first if you want to understand the system

- docs/PRD_v2.md — the rework spec.
- docs/PHASE_1_TASK_MAP.md — frame envelope, env vars, phase labels (foundational contracts).
- server/app_ws.py — the heart of the new transport.
- nao/wake_state.py — the wake state machine.
- nao/utils/brain.py — the local brain cache.
- server/dashboards/README.md — how to bring up Prometheus + Grafana.

Common tasks

Push to GitHub for review

```
git push origin dev/architecture-rework
```

Test the new transport without touching the robot

```
USE_WS=1 ./run.sh server-only
curl http://localhost:5050/health
curl http://localhost:5050/metrics
```

Force the legacy Flask path (rollback)

```
USE_WS=0 ./run.sh
```

Wipe brain cache on robot

```
ssh nao@172.20.95.127 'rm -f ~/nao_assist/brain.json'
```

Apply DB migration

```
python -c 'from server.migrations import apply_pending_migrations; apply_pending_migrations()'
```

Where the agent reports live

Each parallel agent committed with the message format [Phase N] <slug>: <summary>. Search the git log:

```
git log --oneline --grep='\[Phase' main..HEAD
```

Each agent's detailed report (decisions, contract questions, deps declared) lives in the commit body or in their worktree at `.claude/worktrees/agent-<hash>/`.

20. Glossary

NAOqi	Aldebaran/SoftBank middleware. Python 2.7 + C++. Provides ALMotion, ALAudioDevice, ALF
WebSocket	Long-lived bidirectional TCP stream. Replaces HTTP request/response for the voice loop.
FastAPI	Async-first Python web framework. Replaces Flask in this rework.
uvicorn	ASGI server that hosts FastAPI.
VAD	Voice activity detection. Two layers: energy-based on robot, Silero (neural) on server (authorit
Silero	Open-source neural VAD. We use it server-side in streaming mode with adaptive bimodal thre
EoU arbiter	End-of-utterance arbiter. Combines Silero + robot energy hint + semantic_endpoint to decide a
semantic_endpoint	GPT-4o-mini single-token Yes/No classifier on transcript-so-far. Async + LRU+TTL cached.
Realtime API	OpenAI's WebRTC/WebSocket voice API. Considered, benchmarked, NOT chosen (D
WebRTC	Real-time peer-to-peer protocol with FEC and Opus. Python 2.7 has no working WebRTC client
AWARE state	The intermediate wake state we added. Face detected but NOT engaged. Prevents auto-gree
Engagement gate	One of 5 signals that promotes AWARE → ENGAGED: mutual gaze, proximity, sustained face
BrainCache	Robot-side 64 KB JSON cache for identity, preferences, prompt fragments. NOT a knowledge
brain_sync	Server→robot WS frame that pushes brain.json updates.
Motion-trigger shortcut	Regex on transcript that bypasses the LLM for clear body-action commands. Sub-second resp
Crisis gate	Hardcoded 988 hotline reply that runs before any agent. LLM cannot override.
CS Navigator	Operator's deployed Cloud Run FastAPI for Morgan-CS knowledge. Replaces Pinecon
Phase label	A named stage we time with <code>metrics.phase_timer(label)</code> . 22 labels whitelisted;
structlog	Structured JSON logging. Per-turn event with user/session_id/turn_idx/phase_ms/transcript/ou
Prometheus	Time-series metrics collector. We expose /metrics endpoint with 1 histogram + 8 counters.
Grafana	Dashboard tool that queries Prometheus. We ship a 10-panel JSON for import.
Worktree	Isolated git checkout. Each parallel agent gets one off dev/architecture-rework.
Task map	Per-phase contract document at <code>docs/PHASE_N_TASK_MAP.md</code> . Defines file

End of walkthrough

Drop this PDF into NotebookLM as the primary source. Ask it any of:

- “Walk me through the wake state machine.”
- “Why was Realtime API rejected?”
- “What does the EoU arbiter do?”
- “How does brain_sync work?”

- “What’s still pending before merge to main?”

For source-level questions, point NotebookLM at docs/PRD_v2.md + the per-phase task maps as additional sources.